



山东大学

信息科学与工程学院

XXX—XXX 学年第二学期

课程实验报告

课程名称: 数据结构与算法导论

实验名称: 学生信息管理系统

专 业 班 级 通信二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2026 年 5 月 27 日

一、实验目的

1. 加深对抽象数据类型 ADT 栈和队列的理解

二、实验原理

单链表：用于保存所有学生信息，每个学生结点保存一个学生的基本信息和成绩链表。

成绩链表：每个学生可以有多门课程成绩，因此使用链表保存课程名和分数。

哈希表：用学号作为关键字，提高按学号查找学生的效率。

栈：用于保存被删除学生的信息，实现“撤销删除”功能。

快速排序：按照学生平均成绩从高到低排序显示。

模块化设计：main.c 负责菜单调度，student.c 负责学生信息管理，input.c 负责输入校验，academic.c 负责年级、学院、专业、班级选择

三、实验内容

本系统主要实现以下功能：

1. 添加学生信息，包括学号、姓名、年龄、性别、年级、学院、专业、班级、宿舍、手机号和课程成绩。
2. 显示所有学生信息。
3. 按学号查找学生。
4. 修改学生基本信息，支持按学号或姓名定位学生。
5. 删除学生信息，并将删除记录压入撤销栈。
6. 为已有学生添加课程成绩。
7. 按平均成绩排序显示。
8. 统计学生总人数、全体平均成绩、最高平均分和最低平均分。
9. 撤销上一次删除操作。
10. 退出系统并释放动态内存。

四、实验源程序

main.c:程序入口文件。

负责显示主菜单、读取用户选择，并调用对应功能，比如添加学生、显示学生、查找学生、修改学生、删除学生、排序、统计、撤销删除、退出系统。

```
#include <stdio.h>
#include "input.h"
#include "student.h"

void menu()
{
    printf("\n===== \n");
    printf("          学生信息管理系统\n");
```

```

printf("=====\n");
printf("1. 添加学生信息\n");
printf("2. 显示所有学生信息\n");
printf("3. 按学号查找学生\n");
printf("4. 修改学生基本信息\n");
printf("5. 删除学生信息\n");
printf("6. 为已有学生添加课程成绩\n");
printf("7. 按平均成绩排序显示\n");
printf("8. 成绩统计\n");
printf("9. 撤销上一次删除\n");
printf("0. 退出系统\n");
printf("=====\n");
}

int main()
{
    int choice;

    while (1)
    {
        menu();
        choice = readInt("请输入你的选择: ");

        switch (choice)
        {
            case 1:
                addStudent();
                break;
            case 2:
                showStudents();
                break;
            case 3:
                searchStudent();
                break;
            case 4:
                modifyStudent();
                break;
            case 5:
                deleteStudent();
                break;
            case 6:
                addGradeForStudent();
                break;
            case 7:

```

```

        sortByAverageScore();
        break;
    case 8:
        statistics();
        break;
    case 9:
        undoDelete();
        break;
    case 0:
        freeAllStudents();
        freeHashTable();
        freeUndoStack();
        printf("程序已退出，相关内存已释放。\\n");
        return 0;
    default:
        printf("输入有误，请重新选择! \\n");
    }
}
}

```

student.c: 学生管理核心实现文件。

负责真正处理学生数据，包括新增学生、显示学生、按学号查找、修改基本信息、删除学生、撤销删除、添加课程成绩、按平均成绩排序、成绩统计、释放内存等。它主要用到了学生链表、成绩链表、哈希表和撤销栈。

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "common.h"
#include "input.h"
#include "academic.h"
#include "student.h"

static StudentNode *studentHead = NULL;
static HashNode *hashTable[HASH_SIZE] = {NULL};
static UndoNode *undoTop = NULL;
static int studentCount = 0;

static GradeNode *createGradeNode(char subject[], float score);
static void inputGrades(Student *s);
static void showOneStudent(Student s);
static float getAverageScore(Student s);

static unsigned int hashFunc(char id[]);
static void insertHash(char id[], StudentNode *ptr);
static void removeHash(char id[]);

```

```

static StudentNode *searchHash(char id[]);
static StudentNode *selectStudentForModify();
static StudentNode *selectStudentByIdForModify();
static StudentNode *selectStudentByNameForModify();
static int inputModifiedBasicInfo(Student *s);
static void showModifyFieldMenu();
static int modifyOneBasicField(Student *s, int choice);
static void copyBasicInfo(Student *dest, Student src);

static GradeNode *copyGrades(GradeNode *head);
static Student copyStudent(Student s);
static void pushUndo(Student s);
static void freeGrades(GradeNode *head);
static void freeStudent(Student s);

static void quickSort(StudentNode *arr[], int left, int right);

void addStudent()
{
    StudentNode *newNode = (StudentNode *)malloc(sizeof(StudentNode));

    if (newNode == NULL)
    {
        printf("内存申请失败! \n");
        return;
    }

    newNode->data.grades = NULL;
    newNode->next = NULL;

    readStudentId(newNode->data.id);

    if (searchHash(newNode->data.id) != NULL)
    {
        printf("该学号已存在, 添加失败! \n");
        free(newNode);
        return;
    }

    readName(newNode->data.name);

    newNode->data.age = readIntRange("请输入年龄: ", 1, 100);

    chooseGender(newNode->data.gender);
}
    
```

```

        chooseAcademicInfo(&newNode->data);

        readDorm(newNode->data.dorm);

        readPhone(newNode->data.phone);

        printf("\n 开始录入课程成绩。 \n");
        inputGrades(&newNode->data);

        newNode->next = studentHead;
        studentHead = newNode;
        studentCount++;

        insertHash(newNode->data.id, newNode);

        printf("学生信息添加成功! \n");
    }

static void inputGrades(Student *s)
{
    char subject[SUBJECT_LEN];
    float score;

    while (1)
    {
        readString("请输入科目名称, 输入 0 结束成绩录入: ", subject,
SUBJECT_LEN);

        if (strcmp(subject, "0") == 0)
        {
            printf("成绩录入结束。 \n");
            break;
        }

        char prompt[100];
        sprintf(prompt, "请输入 %s 的成绩: ", subject);

        score = readFloatRange(prompt, 0, 100);

        GradeNode *newGrade = createGradeNode(subject, score);

        if (newGrade == NULL)
        {

```

```

        printf("成绩结点创建失败! \n");
        return;
    }

    newGrade->next = s->grades;
    s->grades = newGrade;

    printf("成绩录入成功! \n");
}
}

static GradeNode *createGradeNode(char subject[], float score)
{
    GradeNode *node = (GradeNode *)malloc(sizeof(GradeNode));

    if (node == NULL)
    {
        return NULL;
    }

    strcpy(node->subject, subject);
    node->score = score;
    node->next = NULL;

    return node;
}

void showStudents()
{
    if (studentHead == NULL)
    {
        printf("当前没有学生信息。 \n");
        return;
    }

    StudentNode *p = studentHead;
    int index = 1;

    while (p != NULL)
    {
        printf("\n----- 第 %d 个学生 ----- \n", index);
        showOneStudent(p->data);
        p = p->next;
        index++;
    }
}

```

```

    }
}

static void showOneStudent(Student s)
{
    printf("学号: %s\n", s.id);
    printf("姓名: %s\n", s.name);
    printf("年龄: %d\n", s.age);
    printf("性别: %s\n", s.gender);
    printf("年级: %s\n", s.gradeYear);
    printf("学院: %s\n", s.college);
    printf("专业: %s\n", s.major);
    printf("班级: %s\n", s.className);
    printf("宿舍门牌号: %s\n", s.dorm);
    printf("手机号: %s\n", s.phone);

    if (s.grades == NULL)
    {
        printf("课程成绩: 暂无\n");
    }
    else
    {
        printf("课程成绩: \n");

        GradeNode *g = s.grades;
        while (g != NULL)
        {
            printf("  %s: %.2f\n", g->subject, g->score);
            g = g->next;
        }

        printf("平均成绩: %.2f\n", getAverageScore(s));
    }
}

void searchStudent()
{
    char id[ID_LEN];

    readStudentId(id);

    StudentNode *result = searchHash(id);

    if (result == NULL)

```

```

    {
        printf("未找到该学生信息。\\n");
    }
    else
    {
        printf("查找成功! \\n");
        showOneStudent(result->data);
    }
}

static StudentNode *selectStudentByIdForModify()
{
    char id[100];

    while (1)
    {
        printf("请输入要修改学生的学号（12 位数字，输入 0 取消）：");
        readLine(id, sizeof(id));

        if (strcmp(id, "0") == 0)
        {
            return NULL;
        }

        if (strlen(id) != 12 || !isAllDigits(id))
        {
            printf("输入错误！学号必须是 12 位数字。\\n");
            continue;
        }

        StudentNode *target = searchHash(id);

        if (target == NULL)
        {
            printf("未找到该学生，无法修改! \\n");
        }

        return target;
    }
}

static StudentNode *selectStudentByNameForModify()
{
    char name[NAME_LEN];

```

```

char temp[100];
int count = 0;

while (1)
{
    printf("请输入要修改学生的姓名（输入 0 取消）：");
    readLine(temp, sizeof(temp));

    if (strcmp(temp, "0") == 0)
    {
        return NULL;
    }

    if (!isValidName(temp))
    {
        printf("输入错误！姓名不能为空，且不能包含数字。\\n");
        continue;
    }

    strncpy(name, temp, NAME_LEN - 1);
    name[NAME_LEN - 1] = '\\0';
    break;
}

StudentNode *p = studentHead;

while (p != NULL)
{
    if (strcmp(p->data.name, name) == 0)
    {
        count++;
    }

    p = p->next;
}

if (count == 0)
{
    printf("未找到该姓名对应的学生，无法修改！\\n");
    return NULL;
}

StudentNode **matches = (StudentNode **)malloc(sizeof(StudentNode *) *
count);
    
```

```

    if (matches == NULL)
    {
        printf("内存申请失败, 无法按姓名查找! \n");
        return NULL;
    }

    p = studentHead;
    int index = 0;

    while (p != NULL)
    {
        if (strcmp(p->data.name, name) == 0)
        {
            matches[index++] = p;
        }

        p = p->next;
    }

    if (count == 1)
    {
        StudentNode *target = matches[0];
        printf("已找到学生: %s, 学号: %s。 \n", target->data.name,
target->data.id);
        free(matches);
        return target;
    }

    printf("找到多个同名学生, 请选择要修改的记录: \n");

    for (int i = 0; i < count; i++)
    {
        Student *student = &matches[i]->data;

        printf("%d. 学号: %s, 姓名: %s, 学院: %s, 专业: %s, 班级: %s\n",
            i + 1,
            student->id,
            student->name,
            student->college,
            student->major,
            student->className);
    }

```

```

    int choice = readIntRange("请选择编号（输入 0 取消）：", 0, count);

    if (choice == 0)
    {
        free(matches);
        return NULL;
    }

    StudentNode *target = matches[choice - 1];
    free(matches);
    return target;
}

static StudentNode *selectStudentForModify()
{
    int choice;

    printf("\n 请选择查找方式： \n");
    printf("1. 按学号查找\n");
    printf("2. 按姓名查找\n");
    printf("0. 取消修改\n");

    choice = readIntRange("请输入选项：", 0, 2);

    if (choice == 0)
    {
        return NULL;
    }
    else if (choice == 1)
    {
        return selectStudentByIdForModify();
    }

    return selectStudentByNameForModify();
}

static int inputModifiedBasicInfo(Student *s)
{
    if (!readNameCancelable(s->name))
    {
        return 0;
    }

    s->age = readIntRange("请输入新的年龄（输入 0 取消）：", 0, 100);
}
    
```

```

        if (s->age == 0)
        {
            return 0;
        }

        if (!chooseGenderCancelable(s->gender))
        {
            return 0;
        }

        if (!chooseAcademicInfoCancelable(s))
        {
            return 0;
        }

        if (!readDormCancelable(s->dorm))
        {
            return 0;
        }

        if (!readPhoneCancelable(s->phone))
        {
            return 0;
        }

        return 1;
    }

static void showModifyFieldMenu()
{
    printf("\n 请选择要修改的信息: \n");
    printf("1. 姓名\n");
    printf("2. 年龄\n");
    printf("3. 性别\n");
    printf("4. 年级、学院、专业、班级\n");
    printf("5. 宿舍门牌号\n");
    printf("6. 手机号\n");
    printf("7. 修改全部基本信息\n");
    printf("8. 保存当前全部修改\n");
    printf("0. 取消并退出修改\n");
}

static int modifyOneBasicField(Student *s, int choice)

```

```

{
    Student temp = *s;

    switch (choice)
    {
    case 1:
        if (!readNameCancelable(temp.name))
        {
            return 0;
        }
        strcpy(s->name, temp.name);
        break;
    case 2:
        temp.age = readIntRange("请输入新的年龄（输入 0 取消）：", 0, 100);

        if (temp.age == 0)
        {
            return 0;
        }

        s->age = temp.age;
        break;
    case 3:
        if (!chooseGenderCancelable(temp.gender))
        {
            return 0;
        }
        strcpy(s->gender, temp.gender);
        break;
    case 4:
        if (!chooseAcademicInfoCancelable(&temp))
        {
            return 0;
        }
        strcpy(s->gradeYear, temp.gradeYear);
        strcpy(s->college, temp.college);
        strcpy(s->major, temp.major);
        strcpy(s->className, temp.className);
        break;
    case 5:
        if (!readDormCancelable(temp.dorm))
        {
            return 0;
        }
    }
}
    
```

```

        strcpy(s->dorm, temp.dorm);
        break;
    case 6:
        if (!readPhoneCancelable(temp.phone))
        {
            return 0;
        }
        strcpy(s->phone, temp.phone);
        break;
    case 7:
        if (!inputModifiedBasicInfo(&temp))
        {
            return 0;
        }
        copyBasicInfo(s, temp);
        break;
    default:
        return 0;
    }

    return 1;
}

static void copyBasicInfo(Student *dest, Student src)
{
    strcpy(dest->name, src.name);
    dest->age = src.age;
    strcpy(dest->gender, src.gender);
    strcpy(dest->gradeYear, src.gradeYear);
    strcpy(dest->college, src.college);
    strcpy(dest->major, src.major);
    strcpy(dest->className, src.className);
    strcpy(dest->dorm, src.dorm);
    strcpy(dest->phone, src.phone);
}

void modifyStudent()
{
    StudentNode *target = selectStudentForModify();
    Student temp;
    int choice;

    if (target == NULL)
    {
        printf("已取消修改学生基本信息。\\n");
    }
}

```

```

        return;
    }

    temp = target->data;

    while (1)
    {
        printf("当前待保存的学生信息如下: \n");
        showOneStudent(temp);
        showModifyFieldMenu();

        choice = readIntRange("请输入选项: ", 0, 8);

        if (choice == 0)
        {
            printf("已取消修改学生基本信息, 原信息保持不变。 \n");
            return;
        }

        if (choice == 8)
        {
            copyBasicInfo(&target->data, temp);
            printf("学生基本信息修改已保存! \n");
            return;
        }

        if (modifyOneBasicField(&temp, choice))
        {
            printf("本项信息已修改, 按 8 保存后才会生效。 \n");
        }
        else
        {
            printf("已取消本项修改, 当前待保存信息保持不变。 \n");
        }
    }
}

void deleteStudent()
{
    char id[ID_LEN];

    readStudentId(id);

    StudentNode *cur = studentHead;
    StudentNode *pre = NULL;

```

```

while (cur != NULL && strcmp(cur->data.id, id) != 0)
{
    pre = cur;
    cur = cur->next;
}

if (cur == NULL)
{
    printf("未找到该学生，无法删除！\n");
    return;
}

pushUndo(cur->data);

if (pre == NULL)
{
    studentHead = cur->next;
}
else
{
    pre->next = cur->next;
}

removeHash(id);
freeGrades(cur->data.grades);
free(cur);
studentCount--;

printf("学生信息删除成功！可以通过菜单 9 撤销本次删除。\\n");
}

void addGradeForStudent()
{
    char id[ID_LEN];

    readStudentId(id);

    StudentNode *target = searchHash(id);

    if (target == NULL)
    {
        printf("未找到该学生，无法添加成绩！\\n");
        return;
    }
}
    
```

```

    }

    printf("正在为学生 %s 添加课程成绩。\\n", target->data.name);
    inputGrades(&target->data);
}

void sortByAverageScore()
{
    if (studentCount == 0)
    {
        printf("当前没有学生信息，无法排序。\\n");
        return;
    }

    StudentNode **arr = (StudentNode **)malloc(sizeof(StudentNode *) *
studentCount);

    if (arr == NULL)
    {
        printf("排序数组申请失败! \\n");
        return;
    }

    StudentNode *p = studentHead;
    int i = 0;

    while (p != NULL)
    {
        arr[i++] = p;
        p = p->next;
    }

    quickSort(arr, 0, studentCount - 1);

    printf("\\n 按平均成绩从高到低排序结果: \\n");

    for (i = 0; i < studentCount; i++)
    {
        printf("\\n 第 %d 名: \\n", i + 1);
        showOneStudent(arr[i]->data);
    }

    free(arr);
}

```

```

static void quickSort(StudentNode *arr[], int left, int right)
{
    if (left >= right)
    {
        return;
    }

    int i = left;
    int j = right;
    float pivot = getAverageScore(arr[(left + right) / 2]->data);

    while (i <= j)
    {
        while (getAverageScore(arr[i]->data) > pivot)
        {
            i++;
        }

        while (getAverageScore(arr[j]->data) < pivot)
        {
            j--;
        }

        if (i <= j)
        {
            StudentNode *temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
            i++;
            j--;
        }
    }

    if (left < j)
    {
        quickSort(arr, left, j);
    }

    if (i < right)
    {
        quickSort(arr, i, right);
    }
}
    
```

```

void statistics()
{
    if (studentHead == NULL)
    {
        printf("当前没有学生信息，无法统计。\\n");
        return;
    }

    StudentNode *p = studentHead;
    float sum = 0;
    float maxAvg = getAverageScore(p->data);
    float minAvg = getAverageScore(p->data);
    StudentNode *maxNode = p;
    StudentNode *minNode = p;

    while (p != NULL)
    {
        float avg = getAverageScore(p->data);
        sum += avg;

        if (avg > maxAvg)
        {
            maxAvg = avg;
            maxNode = p;
        }

        if (avg < minAvg)
        {
            minAvg = avg;
            minNode = p;
        }

        p = p->next;
    }

    printf("学生总人数: %d\\n", studentCount);
    printf("全体学生平均成绩: %.2f\\n", sum / studentCount);

    printf("平均成绩最高: %s, 学号: %s, 平均分: %.2f\\n",
        maxNode->data.name,
        maxNode->data.id,
        maxAvg);
}
    
```

```

        printf("平均成绩最低: %s, 学号: %s, 平均分: %.2f\n",
               minNode->data.name,
               minNode->data.id,
               minAvg);
    }

void undoDelete()
{
    if (undoTop == NULL)
    {
        printf("当前没有可撤销的删除操作。 \n");
        return;
    }

    if (searchHash(undoTop->data.id) != NULL)
    {
        printf("该学号当前已存在, 无法恢复。 \n");
        return;
    }

    UndoNode *temp = undoTop;
    undoTop = undoTop->next;

    StudentNode *newNode = (StudentNode *)malloc(sizeof(StudentNode));

    if (newNode == NULL)
    {
        printf("恢复失败, 内存不足! \n");
        freeStudent(temp->data);
        free(temp);
        return;
    }

    newNode->data = temp->data;
    newNode->next = studentHead;
    studentHead = newNode;
    studentCount++;

    insertHash(newNode->data.id, newNode);

    free(temp);

    printf("撤销删除成功, 学生信息已恢复。 \n");
}

```

```

static float getAverageScore(Student s)
{
    GradeNode *p = s.grades;
    float sum = 0;
    int count = 0;

    while (p != NULL)
    {
        sum += p->score;
        count++;
        p = p->next;
    }

    if (count == 0)
    {
        return 0;
    }

    return sum / count;
}

static unsigned int hashFunc(char id[])
{
    unsigned int hash = 0;

    for (int i = 0; id[i] != '\0'; i++)
    {
        hash = hash * 31 + id[i];
    }

    return hash % HASH_SIZE;
}

static void insertHash(char id[], StudentNode *ptr)
{
    unsigned int index = hashFunc(id);

    HashNode *node = (HashNode *)malloc(sizeof(HashNode));

    if (node == NULL)
    {
        printf("哈希表结点申请失败! \n");
        return;
    }
}
    
```

```

    }

    strcpy(node->id, id);
    node->studentPtr = ptr;
    node->next = hashTable[index];
    hashTable[index] = node;
}

static void removeHash(char id[])
{
    unsigned int index = hashFunc(id);

    HashNode *cur = hashTable[index];
    HashNode *pre = NULL;

    while (cur != NULL)
    {
        if (strcmp(cur->id, id) == 0)
        {
            if (pre == NULL)
            {
                hashTable[index] = cur->next;
            }
            else
            {
                pre->next = cur->next;
            }

            free(cur);
            return;
        }

        pre = cur;
        cur = cur->next;
    }
}

static StudentNode *searchHash(char id[])
{
    unsigned int index = hashFunc(id);

    HashNode *p = hashTable[index];

    while (p != NULL)

```

```

    {
        if (strcmp(p->id, id) == 0)
        {
            return p->studentPtr;
        }

        p = p->next;
    }

    return NULL;
}

static GradeNode *copyGrades(GradeNode *head)
{
    if (head == NULL)
    {
        return NULL;
    }

    GradeNode *newHead = NULL;
    GradeNode *tail = NULL;

    while (head != NULL)
    {
        GradeNode *newNode = createGradeNode(head->subject, head->score);

        if (newNode == NULL)
        {
            freeGrades(newHead);
            return NULL;
        }

        if (newHead == NULL)
        {
            newHead = newNode;
            tail = newNode;
        }
        else
        {
            tail->next = newNode;
            tail = newNode;
        }

        head = head->next;
    }
}

```

```

    }

    return newHead;
}

static Student copyStudent(Student s)
{
    Student copy = s;
    copy.grades = copyGrades(s.grades);
    return copy;
}

static void pushUndo(Student s)
{
    UndoNode *node = (UndoNode *)malloc(sizeof(UndoNode));

    if (node == NULL)
    {
        printf("撤销栈空间不足，本次删除不可撤销。\\n");
        return;
    }

    node->data = copyStudent(s);
    node->next = undoTop;
    undoTop = node;
}

static void freeGrades(GradeNode *head)
{
    GradeNode *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }
}

static void freeStudent(Student s)
{
    freeGrades(s.grades);
}

```

```

void freeAllStudents()
{
    StudentNode *cur = studentHead;
    StudentNode *temp;

    while (cur != NULL)
    {
        temp = cur;
        cur = cur->next;
        freeGrades(temp->data.grades);
        free(temp);
    }

    studentHead = NULL;
    studentCount = 0;
}

void freeHashTable()
{
    for (int i = 0; i < HASH_SIZE; i++)
    {
        HashNode *cur = hashTable[i];
        HashNode *temp;

        while (cur != NULL)
        {
            temp = cur;
            cur = cur->next;
            free(temp);
        }

        hashTable[i] = NULL;
    }
}

void freeUndoStack()
{
    UndoNode *cur = undoTop;
    UndoNode *temp;

    while (cur != NULL)
    {
        temp = cur;
        cur = cur->next;
    }
}
    
```

```

        freeStudent(temp->data);
        free(temp);
    }

    undoTop = NULL;
}

```

input.c 输入和校验实现文件。

负责统一处理用户输入，保证输入格式正确。比如学号必须是 12 位数字，手机号必须是 11 位数字，年龄必须在指定范围内，姓名不能为空且不能包含数字。

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include "input.h"

void readLine(char *buffer, int size)
{
    if (fgets(buffer, size, stdin) == NULL)
    {
        buffer[0] = '\0';
        return;
    }

    size_t len = strlen(buffer);
    if (len > 0 && buffer[len - 1] == '\n')
    {
        buffer[len - 1] = '\0';
    }
    else
    {
        int ch;

        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }
    }
}

int readInt(const char *prompt)
{
    char line[100];
    int value;
    char extra;

    while (1)

```

```

    {
        printf("%s", prompt);
        readLine(line, sizeof(line));

        if (sscanf(line, "%d %c", &value, &extra) == 1)
        {
            return value;
        }

        printf("输入错误! 请输入整数。\\n");
    }
}

int readIntRange(const char *prompt, int min, int max)
{
    int value;

    while (1)
    {
        value = readInt(prompt);

        if (value >= min && value <= max)
        {
            return value;
        }

        printf("输入错误! 请输入 %d 到 %d 之间的整数。\\n", min, max);
    }
}

int readIntRangeCancelable(const char *prompt, int min, int max)
{
    int value;

    while (1)
    {
        value = readInt(prompt);

        if (value == 0)
        {
            return 0;
        }
    }
}

```

```

        if (value >= min && value <= max)
        {
            return value;
        }

        printf("输入错误! 请输入 %d 到 %d 之间的整数, 或输入 0 取消.\n", min,
max);
    }
}

float readFloatRange(const char *prompt, float min, float max)
{
    char line[100];
    float value;
    char extra;

    while (1)
    {
        printf("%s", prompt);
        readLine(line, sizeof(line));

        if (sscanf(line, "%f %c", &value, &extra) == 1)
        {
            if (value >= min && value <= max)
            {
                return value;
            }

            printf("输入错误! 数值范围应为 %.1f 到 %.1f.\n", min, max);
        }
        else
        {
            printf("输入错误! 请输入数字, 例如: 85、92.5.\n");
        }
    }
}

void readString(const char *prompt, char *buffer, int size)
{
    while (1)
    {
        printf("%s", prompt);
        readLine(buffer, size);

        if (strlen(buffer) > 0)
    
```

```

        {
            return;
        }

        printf("输入不能为空，请重新输入。 \n");
    }
}

int isAllDigits(const char *str)
{
    if (strlen(str) == 0)
    {
        return 0;
    }

    for (int i = 0; str[i] != '\0'; i++)
    {
        if (!isdigit((unsigned char)str[i]))
        {
            return 0;
        }
    }

    return 1;
}

void readStudentId(char *id)
{
    char temp[100];

    while (1)
    {
        printf("请输入学号（12 位数字）： ");
        readLine(temp, sizeof(temp));

        if (strlen(temp) == 12 && isAllDigits(temp))
        {
            strcpy(id, temp);
            return;
        }

        printf("输入错误！学号必须是 12 位数字，例如：202500120055。 \n");
    }
}

```

```

int isValidName(const char *str)
{
    if (strlen(str) == 0)
    {
        return 0;
    }

    for (int i = 0; str[i] != '\0'; i++)
    {
        if (isdigit((unsigned char)str[i]))
        {
            return 0;
        }
    }

    return 1;
}

void readName(char *name)
{
    char temp[100];

    while (1)
    {
        printf("请输入姓名: ");
        readLine(temp, sizeof(temp));

        if (isValidName(temp))
        {
            strncpy(name, temp, NAME_LEN - 1);
            name[NAME_LEN - 1] = '\0';
            return;
        }

        printf("输入错误! 姓名不能为空, 且不能包含数字.\n");
    }
}

int readNameCancelable(char *name)
{
    char temp[100];

```

```

while (1)
{
    printf("请输入新的姓名（输入 0 取消）：");
    readLine(temp, sizeof(temp));

    if (strcmp(temp, "0") == 0)
    {
        return 0;
    }

    if (isValidName(temp))
    {
        strncpy(name, temp, NAME_LEN - 1);
        name[NAME_LEN - 1] = '\0';
        return 1;
    }

    printf("输入错误！姓名不能为空，且不能包含数字。\\n");
}
}

void readPhone(char *phone)
{
    char temp[100];

    while (1)
    {
        printf("请输入手机号（11 位数字）：");
        readLine(temp, sizeof(temp));

        if (strlen(temp) == 11 && isAllDigits(temp))
        {
            strcpy(phone, temp);
            return;
        }

        printf("输入错误！手机号必须是 11 位数字。\\n");
    }
}

int readPhoneCancelable(char *phone)
{
    char temp[100];

```

```

while (1)
{
    printf("请输入新的手机号（11 位数字，输入 0 取消）：");
    readLine(temp, sizeof(temp));

    if (strcmp(temp, "0") == 0)
    {
        return 0;
    }

    if (strlen(temp) == 11 && isAllDigits(temp))
    {
        strcpy(phone, temp);
        return 1;
    }

    printf("输入错误！手机号必须是 11 位数字。\\n");
}
}

void readDorm(char *dorm)
{
    readString("请输入宿舍门牌号：", dorm, DORM_LEN);
}

int readDormCancelable(char *dorm)
{
    char temp[100];

    while (1)
    {
        printf("请输入新的宿舍门牌号（输入 0 取消）：");
        readLine(temp, sizeof(temp));

        if (strcmp(temp, "0") == 0)
        {
            return 0;
        }

        if (strlen(temp) > 0)
        {
            strncpy(dorm, temp, DORM_LEN - 1);
            dorm[DORM_LEN - 1] = '\\0';
            return 1;
        }
    }
}
    
```

```

    }

    printf("输入不能为空，请重新输入。 \n");
}
}

void chooseGender(char *gender)
{
    int choice;

    while (1)
    {
        printf("请选择性别: \n");
        printf("1. 男\n");
        printf("2. 女\n");

        choice = readInt("请输入选项: ");

        if (choice == 1)
        {
            strcpy(gender, "男");
            return;
        }
        else if (choice == 2)
        {
            strcpy(gender, "女");
            return;
        }

        printf("输入错误！性别只能选择 1 或 2。 \n");
    }
}

int chooseGenderCancellable(char *gender)
{
    int choice;

    while (1)
    {
        printf("请选择新的性别: \n");
        printf("1. 男\n");
        printf("2. 女\n");
        printf("0. 取消修改\n");

        choice = readIntRange("请输入选项: ", 0, 2);
    }
}

```

```

        if (choice == 0)
        {
            return 0;
        }
        else if (choice == 1)
        {
            strcpy(gender, "男");
            return 1;
        }
        else if (choice == 2)
        {
            strcpy(gender, "女");
            return 1;
        }
    }
}

```

academic.c 学籍信息实现文件。

负责选择年级、学院、专业、班级。它里面保存了学院列表和各学院对应的专业列表。新增或修改学生时，会调用这里的函数来填写学籍信息。

```

#include <stdio.h>
#include <string.h>
#include "academic.h"
#include "input.h"

static const char *colleges[] =
{
    "信息科学与工程学院",
    "计算机科学与技术学院",
    "网络空间安全学院",
    "生命科学学院",
    "环境科学与工程学院",
    "法学院",
    "政治学与公共管理学院"
};

static const char *infoMajors[] =
{
    "通信工程",
    "电子科学与技术",
    "光电信息科学与工程",
    "崇信学堂",
    "微纳光子新工科实验班",
    "光电科创精英班"
};

```

```
static const char *csMajors[] =
{
    "计算机科学与技术",
    "人工智能",
    "数据科学与大数据技术"
};

static const char *cyberMajors[] =
{
    "网络空间安全",
    "密码科学与技术"
};

static const char *lifeMajors[] =
{
    "生物科学",
    "生物技术",
    "生物工程"
};

static const char *envMajors[] =
{
    "环境工程",
    "环境科学"
};

static const char *lawMajors[] =
{
    "法学"
};

static const char *politicsMajors[] =
{
    "政治学与经济学",
    "国际政治",
    "行政管理"
};

static void chooseGrade(char *grade)
{
    int choice;
```

```

printf("请选择年级: \n");
printf("1. 大一\n");
printf("2. 大二\n");
printf("3. 大三\n");
printf("4. 大四\n");

choice = readIntRange("请输入选项: ", 1, 4);

if (choice == 1)
{
    strcpy(grade, "大一");
}
else if (choice == 2)
{
    strcpy(grade, "大二");
}
else if (choice == 3)
{
    strcpy(grade, "大三");
}
else
{
    strcpy(grade, "大四");
}
}

static int chooseCollege(char *college)
{
    printf("请选择学院: \n");

    for (int i = 0; i < 7; i++)
    {
        printf("%d. %s\n", i + 1, colleges[i]);
    }

    int choice = readIntRange("请输入选项: ", 1, 7);

    strcpy(college, colleges[choice - 1]);

    return choice;
}

static void chooseMajorFromList(char *major, const char *majors[], int
count)

```

```

{
    printf("请选择专业: \n");

    for (int i = 0; i < count; i++)
    {
        printf("%d. %s\n", i + 1, majors[i]);
    }

    int choice = readIntRange("请输入选项: ", 1, count);

    strcpy(major, majors[choice - 1]);
}

static void chooseMajorByCollege(int collegeChoice, char *major)
{
    switch (collegeChoice)
    {
        case 1:
            chooseMajorFromList(major, infoMajors, 6);
            break;
        case 2:
            chooseMajorFromList(major, csMajors, 3);
            break;
        case 3:
            chooseMajorFromList(major, cyberMajors, 2);
            break;
        case 4:
            chooseMajorFromList(major, lifeMajors, 3);
            break;
        case 5:
            chooseMajorFromList(major, envMajors, 2);
            break;
        case 6:
            chooseMajorFromList(major, lawMajors, 1);
            break;
        case 7:
            chooseMajorFromList(major, politicsMajors, 3);
            break;
        default:
            strcpy(major, "未知专业");
    }
}

static int noClassNeeded(const char *major)

```

```

{
    return strcmp(major, "崇信学堂") == 0 ||
           strcmp(major, "微纳光电子新工科实验班") == 0 ||
           strcmp(major, "光电科创精英班") == 0;
}

static void inputClassName(char *className, const char *major)
{
    if (noClassNeeded(major))
    {
        strcpy(className, "无需填写");
        return;
    }

    int classNumber;

    classNumber = readIntRange("请输入班级编号,例如输入 1 表示 1 班:", 1, 99);

    sprintf(className, "%d 班", classNumber);
}

void chooseAcademicInfo(Student *s)
{
    chooseGrade(s->gradeYear);

    int collegeChoice = chooseCollege(s->college);

    if (strcmp(s->gradeYear, "大一") == 0)
    {
        strcpy(s->major, "大一暂未分专业");
        strcpy(s->className, "大一暂未分班");
        return;
    }

    chooseMajorByCollege(collegeChoice, s->major);

    inputClassName(s->className, s->major);
}

static int chooseGradeCancelable(char *grade)
{
    int choice;

    printf("请选择新的年级: \n");
    printf("1. 大一\n");

```

```

printf("2. 大二\n");
printf("3. 大三\n");
printf("4. 大四\n");
printf("0. 取消修改\n");

choice = readIntRange("请输入选项: ", 0, 4);

if (choice == 0)
{
    return 0;
}
else if (choice == 1)
{
    strcpy(grade, "大一");
}
else if (choice == 2)
{
    strcpy(grade, "大二");
}
else if (choice == 3)
{
    strcpy(grade, "大三");
}
else
{
    strcpy(grade, "大四");
}

return 1;
}

static int chooseCollegeCancelable(char *college)
{
    printf("请选择新的学院: \n");

    for (int i = 0; i < 7; i++)
    {
        printf("%d. %s\n", i + 1, colleges[i]);
    }

    printf("0. 取消修改\n");

    int choice = readIntRange("请输入选项: ", 0, 7);

```

```

        if (choice == 0)
        {
            return 0;
        }

        strcpy(college, colleges[choice - 1]);

        return choice;
    }

static int chooseMajorFromListCancelable(char *major, const char *majors[],
int count)
{
    printf("请选择新的专业: \n");

    for (int i = 0; i < count; i++)
    {
        printf("%d. %s\n", i + 1, majors[i]);
    }

    printf("0. 取消修改\n");

    int choice = readIntRange("请输入选项: ", 0, count);

    if (choice == 0)
    {
        return 0;
    }

    strcpy(major, majors[choice - 1]);
    return 1;
}

static int chooseMajorByCollegeCancelable(int collegeChoice, char *major)
{
    switch (collegeChoice)
    {
    case 1:
        return chooseMajorFromListCancelable(major, infoMajors, 6);
    case 2:
        return chooseMajorFromListCancelable(major, csMajors, 3);
    case 3:
        return chooseMajorFromListCancelable(major, cyberMajors, 2);
    case 4:

```

```

        return chooseMajorFromListCancelable(major, lifeMajors, 3);
    case 5:
        return chooseMajorFromListCancelable(major, envMajors, 2);
    case 6:
        return chooseMajorFromListCancelable(major, lawMajors, 1);
    case 7:
        return chooseMajorFromListCancelable(major, politicsMajors, 3);
    default:
        strcpy(major, "未知专业");
        return 1;
    }
}

static int inputClassNameCancelable(char *className, const char *major)
{
    if (noClassNeeded(major))
    {
        strcpy(className, "无需填写");
        return 1;
    }

    int classNumber = readIntRange("请输入新的班级编号，例如输入 1 表示 1 班；
    输入 0 取消：", 0, 99);

    if (classNumber == 0)
    {
        return 0;
    }

    sprintf(className, "%d 班", classNumber);
    return 1;
}

int chooseAcademicInfoCancelable(Student *s)
{
    if (!chooseGradeCancelable(s->gradeYear))
    {
        return 0;
    }

    int collegeChoice = chooseCollegeCancelable(s->college);

    if (collegeChoice == 0)
    {

```

```

        return 0;
    }

    if (strcmp(s->gradeYear, "大一") == 0)
    {
        strcpy(s->major, "大一暂未分专业");
        strcpy(s->className, "大一暂未分班");
        return 1;
    }

    if (!chooseMajorByCollegeCancelable(collegeChoice, s->major))
    {
        return 0;
    }

    if (!inputClassNameCancelable(s->className, s->major))
    {
        return 0;
    }

    return 1;
}

```

student.h 学生管理模块的头文件。

声明了学生管理相关函数，例如 addStudent()、showStudents()、searchStudent()、modifyStudent()、deleteStudent()、sortByAverageScore() 等，方便 main.c 调用。

```

#ifndef STUDENT_H
#define STUDENT_H

void addStudent();
void showStudents();
void searchStudent();
void modifyStudent();
void deleteStudent();
void addGradeForStudent();
void sortByAverageScore();
void statistics();
void undoDelete();

void freeAllStudents();
void freeHashTable();
void freeUndoStack();

#endif

```

input.h 输入模块的头文件。

声明各种输入函数，比如读取整数、读取范围内整数、读取姓名、读取学号、读取手机号、选择性别等。

```
#ifndef INPUT_H
#define INPUT_H

#include "common.h"

void readLine(char *buffer, int size);
int readInt(const char *prompt);
int readIntRange(const char *prompt, int min, int max);
int readIntRangeCancelable(const char *prompt, int min, int max);
float readFloatRange(const char *prompt, float min, float max);

void readString(const char *prompt, char *buffer, int size);
void readStudentId(char *id);
void readName(char *name);
int readNameCancelable(char *name);
void readPhone(char *phone);
int readPhoneCancelable(char *phone);
void readDorm(char *dorm);
int readDormCancelable(char *dorm);
void chooseGender(char *gender);
int chooseGenderCancelable(char *gender);

int isAllDigits(const char *str);
int isValidName(const char *str);

#endif
```

academic.h 学籍信息模块的头文件。

声明录入和修改学籍信息的函数

```
#ifndef ACADEMIC_H
#define ACADEMIC_H

#include "common.h"

void chooseAcademicInfo(Student *s);
int chooseAcademicInfoCancelable(Student *s);

#endif
```

common.h 公共头文件。

里面定义了整个系统都会用到的常量和结构体，比如学生信息 Student、成绩结点 GradeNode、学生链表结点 StudentNode、哈希表结点 HashNode、撤销栈结点 UndoNode。

```
#ifndef COMMON_H
```

```

#define COMMON_H

#define ID_LEN 13
#define NAME_LEN 30
#define GENDER_LEN 10
#define GRADE_LEN 20
#define COLLEGE_LEN 80
#define MAJOR_LEN 80
#define CLASS_LEN 30
#define DORM_LEN 30
#define PHONE_LEN 12
#define SUBJECT_LEN 30
#define HASH_SIZE 101

typedef struct GradeNode GradeNode;
typedef struct Student Student;
typedef struct StudentNode StudentNode;
typedef struct HashNode HashNode;
typedef struct UndoNode UndoNode;

struct GradeNode
{
    char subject[SUBJECT_LEN];
    float score;
    GradeNode *next;
};

struct Student
{
    char id[ID_LEN];
    char name[NAME_LEN];
    int age;
    char gender[GENDER_LEN];
    char gradeYear[GRADE_LEN];
    char college[COLLEGE_LEN];
    char major[MAJOR_LEN];
    char className[CLASS_LEN];
    char dorm[DORM_LEN];
    char phone[PHONE_LEN];
    GradeNode *grades;
};

struct StudentNode
{

```

```

        Student data;
        StudentNode *next;
    };

    struct HashNode
    {
        char id[ID_LEN];
        StudentNode *studentPtr;
        HashNode *next;
    };

    struct UndoNode
    {
        Student data;
        UndoNode *next;
    };

#endif
    
```

六、实验结果

(一)主菜单设计

核心代码：main.c

说明：

该部分通过循环显示菜单，并根据用户输入调用不同功能函数。例如输入 1 调用 addStudent(), 输入 2 调用 showStudents(), 输入 0 时释放内存并退出系统。

运行结果：

```

=====
                学生信息管理系统
=====
1. 添加学生信息
2. 显示所有学生信息
3. 按学号查找学生
4. 修改学生基本信息
5. 删除学生信息
6. 为已有学生添加课程成绩
7. 按平均成绩排序显示
8. 成绩统计
9. 撤销上一次删除
0. 退出系统
=====
请输入你的选择：1
请输入学号（12位数字）：
    
```

(二) 添加学生信息

核心代码：student.c 的 addstudent()

```
void addStudent()
{
    StudentNode *newNode = (StudentNode *)malloc(sizeof(StudentNode));

    ...

    readStudentId(newNode->data.id);

    ...

    chooseGender(newNode->data.gender);

    ...
}
```

添加学生姓名，年龄，性别，年级，学院，专业，班级，宿舍门牌号，手机号，课程名称，课程成绩（其中年级，学院，专业和班级信息需要调用 `academic.c` 的函数）

其中

```
readStudentId(newNode->data.id);

if (searchHash(newNode->data.id) != NULL)
{
    printf("该学号已存在，添加失败！\n");
    free(newNode);
    return;
}
```

该代码先读取学号，再通过哈希表判断学号是否已经存在，避免重复添加学生。运行结果如下：

```
=====
请输入你的选择: 1
请输入学号 (12位数字): 202400120045
请输入姓名: 刘哲旭
请输入年龄: 20
请选择性别:
1. 男
2. 女
请输入选项: 1
请选择年级:
1. 大一
2. 大二
3. 大三
4. 大四
请输入选项: 2
请选择学院:
1. 信息科学与工程学院
2. 计算机科学与技术学院
3. 网络空间安全学院
4. 生命科学学院
5. 环境科学与工程学院
6. 法学院
7. 政治学与公共管理学院
请输入选项: 1
请选择专业:
1. 通信工程
2. 电子科学与技术
3. 光电信息科学与工程
4. 崇信学堂
5. 微纳光电子新工科实验班
6. 光电科创精英班
请输入选项: 1
请输入班级编号, 例如输入 1 表示 1班: 2
请输入宿舍门牌号: S1B207
请输入手机号 (11位数字): 15765477177
```

```
开始录入课程成绩。
请输入科目名称, 输入 0 结束成绩录入: 高频电子线路
请输入 高频电子线路 的成绩: 100
成绩录入成功!
请输入科目名称, 输入 0 结束成绩录入: 随机信号分析
请输入 随机信号分析 的成绩: 99
成绩录入成功!
请输入科目名称, 输入 0 结束成绩录入: FPGA
请输入 FPGA 的成绩: 95
成绩录入成功!
请输入科目名称, 输入 0 结束成绩录入: 数字信号处理
请输入 数字信号处理 的成绩: 98
成绩录入成功!
请输入科目名称, 输入 0 结束成绩录入: 0
成绩录入结束。
学生信息添加成功!
```

然后在添加两个名字相同的学生的信息，便于后续功能展示

```
=====
请输入你的选择：1
请输入学号（12位数字）：202500120055
请输入姓名：小明
请输入年龄：19
请选择性别：
1. 男
2. 女
请输入选项：1
请选择年级：
1. 大一
2. 大二
3. 大三
4. 大四
请输入选项：1
请选择学院：
1. 信息科学与工程学院
2. 计算机科学与技术学院
3. 网络空间安全学院
4. 生命科学学院
5. 环境科学与工程学院
6. 法学院
7. 政治学与公共管理学院
请输入选项：1
请输入宿舍门牌号：S1B411
请输入手机号（11位数字）：15845447777
```

```
开始录入课程成绩。
请输入科目名称，输入 0 结束成绩录入：高等数学
请输入 高等数学 的成绩：95
成绩录入成功！
请输入科目名称，输入 0 结束成绩录入：大学物理
请输入 大学物理 的成绩：87
成绩录入成功！
请输入科目名称，输入 0 结束成绩录入：大学英语
请输入 大学英语 的成绩：100
成绩录入成功！
请输入科目名称，输入 0 结束成绩录入：体育
请输入 体育 的成绩：99
成绩录入成功！
请输入科目名称，输入 0 结束成绩录入：0
成绩录入结束。
学生信息添加成功！
```

=====

请输入你的选择: 1
 请输入学号 (12位数字): 202400130013
 请输入姓名: 小明
 请输入年龄: 20
 请选择性别:
 1. 男
 2. 女
 请输入选项: 1
 请选择年级:
 1. 大一
 2. 大二
 3. 大三
 4. 大四
 请输入选项: 2
 请选择学院:
 1. 信息科学与工程学院
 2. 计算机科学与技术学院
 3. 网络空间安全学院
 4. 生命科学学院
 5. 环境科学与工程学院
 6. 法学院
 7. 政治学与公共管理学院
 请输入选项: 2
 请选择专业:
 1. 计算机科学与技术
 2. 人工智能
 3. 数据科学与大数据技术
 请输入选项: 1
 请输入班级编号, 例如输入 1 表示 1班: 2
 请输入宿舍门牌号: S2B115
 请输入手机号 (11位数字): 18756559899

开始录入课程成绩。
 请输入科目名称, 输入 0 结束成绩录入: 计算机组成原理
 请输入 计算机组成原理 的成绩: 78
 成绩录入成功!
 请输入科目名称, 输入 0 结束成绩录入: 数字电子技术
 请输入 数字电子技术 的成绩: 99
 成绩录入成功!
 请输入科目名称, 输入 0 结束成绩录入: 大学物理
 请输入 大学物理 的成绩: 92
 成绩录入成功!
 请输入科目名称, 输入 0 结束成绩录入: 数据结构
 请输入 数据结构 的成绩: 96
 成绩录入成功!
 请输入科目名称, 输入 0 结束成绩录入: 0
 成绩录入结束。
 学生信息添加成功!

(三) 显示学生信息

核心代码: student.c 的 showStudents() 和 showOneStudent()

运行结果如下

请输入你的选择：2

----- 第 1 个学生 -----

学号：202400130013

姓名：小明

年龄：20

性别：男

年级：大二

学院：计算机科学与技术学院

专业：计算机科学与技术

班级：2班

宿舍门牌号：S2B115

手机号：18756559899

课程成绩：

数据结构：96.00

大学物理：92.00

数字电子技术：99.00

计算机组成原理：78.00

平均成绩：91.25

----- 第 2 个学生 -----

学号：202500120055

姓名：小明

年龄：19

性别：男

年级：大一

学院：信息科学与工程学院

专业：大一暂未分专业

班级：大一暂未分班

宿舍门牌号：S1B411

手机号：15845447777

课程成绩：

体育：99.00

大学英语：100.00

大学物理：87.00

高等数学：95.00

平均成绩：95.25

----- 第 3 个学生 -----

学号：202400120045

姓名：刘哲旭

年龄：20

性别：男

年级：大二

学院：信息科学与工程学院

专业：通信工程

班级：2班

宿舍门牌号：S1B207

手机号：15765477177

课程成绩：

数字信号处理：98.00

FPGA：95.00

随机信号分析：99.00

高频电子线路：100.00

（四）按照查找学生

核心代码：

student.c 的 searchStudent(), 再配合 student.c 的 searchHash()。

说明：

用户输入学号后，系统通过哈希函数定位到哈希表中的对应位置，再在冲突链表中查找对应学生，提高查找效率。

运行结果如下：

```
=====
请输入你的选择：3
请输入学号（12位数字）：202400120045
查找成功！
学号：202400120045
姓名：刘哲旭
年龄：20
性别：男
年级：大二
学院：信息科学与工程学院
专业：通信工程
班级：2班
宿舍门牌号：S1B207
手机号：15765477177
课程成绩：
    数字信号处理：98.00
    FPGA：95.00
    随机信号分析：99.00
    高频电子线路：100.00
平均成绩：98.00
```

（五）修改学生信息

核心代码：student.c 中 modifyStudent() 的代码。

说明：

系统先选择要修改的学生，再将原信息复制到临时变量 temp 中。用户修改后只有选择“保存”才会写回原学生结点，这样可以避免误操作。

运行结果如下

=====

请输入你的选择：4

请选择查找方式：

1. 按学号查找
2. 按姓名查找
0. 取消修改

请输入选项：1

请输入要修改学生的学号（12位数字，输入 0 取消）：202500120055

当前待保存的学生信息如下：

学号：202500120055

姓名：小明

年龄：19

性别：男

年级：大一

学院：信息科学与工程学院

专业：大一暂未分专业

班级：大一暂未分班

宿舍门牌号：S1B411

手机号：15845447777

课程成绩：

体育：99.00

大学英语：100.00

大学物理：87.00

高等数学：95.00

平均成绩：95.25

请选择要修改的信息：

1. 姓名
2. 年龄
3. 性别
4. 年级、学院、专业、班级
5. 宿舍门牌号
6. 手机号
7. 修改全部基本信息
8. 保存当前全部修改
0. 取消并退出修改

请输入选项：6

请输入新的手机号（11位数字，输入 0 取消）：1578769898

输入错误！手机号必须是 11 位数字。

请输入新的手机号（11位数字，输入 0 取消）：15787659898

本项信息已修改，按 8 保存后才会生效。

当前待保存的学生信息如下：

学号：202500120055

姓名：小明

年龄：19

性别：男

年级：大一

学院：信息科学与工程学院

专业：大一暂未分专业

班级：大一暂未分班

宿舍门牌号：S1B411

手机号：15787659898

这次使用姓名方式查找小明的个人信息，由于有两个相同姓名的人，系统会输出重名的人的具体信息，然后进一步选择

```
=====
请输入你的选择: 4

请选择查找方式:
1. 按学号查找
2. 按姓名查找
0. 取消修改
请输入选项: 2
请输入要修改学生的姓名(输入 0 取消): 小明
找到多个同名学生, 请选择要修改的记录:
1. 学号: 202400130013, 姓名: 小明, 学院: 计算机科学与技术学院, 专业: 计算机科学与技术, 班级: 2班
2. 学号: 202500120055, 姓名: 小明, 学院: 信息科学与工程学院, 专业: 大一暂未分专业, 班级: 大一暂未分班
请选择编号(输入 0 取消): 1
当前待保存的学生信息如下:
学号: 202400130013
姓名: 小明
年龄: 20
性别: 男
年级: 大二
学院: 计算机科学与技术学院
专业: 计算机科学与技术
班级: 2班
宿舍门牌号: S2B115
手机号: 18756559899
课程成绩:
    数据结构: 96.00
    大学物理: 92.00
    数字电子技术: 99.00
    计算机组成原理: 78.00
平均成绩: 91.25

请选择要修改的信息:
1. 姓名
2. 年龄
3. 性别
4. 年级、学院、专业、班级
5. 宿舍门牌号
6. 手机号
7. 修改全部基本信息
8. 保存当前全部修改
0. 取消并退出修改
请输入选项: 4
请选择新的年级:
1. 大一
2. 大二
3. 大三
4. 大四
0. 取消修改
请输入选项: 2
请选择新的学院:
1. 信息科学与工程学院
2. 计算机科学与技术学院
3. 网络空间安全学院
4. 生命科学学院
```

```

2. 计算机科学与技术学院
3. 网络空间安全学院
4. 生命科学学院
5. 环境科学与工程学院
6. 法学院
7. 政治学与公共管理学院
0. 取消修改
请输入选项：1
请选择新的专业：
1. 通信工程
2. 电子科学与技术
3. 光电信息科学与工程
4. 崇信学堂
5. 微纳光电子新工科实验班
6. 光电科创精英班
0. 取消修改
请输入选项：4
本项信息已修改，按 8 保存后才会生效。
当前待保存的学生信息如下：
学号：202400130013
姓名：小明
年龄：20
性别：男
年级：大二
学院：信息科学与工程学院
专业：崇信学堂
班级：无需填写
宿舍门牌号：S2B115
手机号：18756559899
课程成绩：
    数据结构：96.00
    大学物理：92.00
    数字电子技术：99.00
    计算机组成原理：78.00
平均成绩：91.25

请选择要修改的信息：
1. 姓名
2. 年龄
3. 性别
4. 年级、学院、专业、班级
5. 宿舍门牌号
6. 手机号
7. 修改全部基本信息
8. 保存当前全部修改
0. 取消并退出修改
请输入选项：8
学生基本信息修改已保存！
    
```

（六）删除学生与撤销删除

核心代码：

删除功能贴 student.c 的 deleteStudent()。

撤销功能贴 student.c 的 undoDelete()。

删除前调用 pushUndo(cur->data) 保存被删除学生信息，删除后从链表和哈希表中移除该学生。撤销时再从撤销栈中取出最近一次删除的学生并恢复到链表中。运行结果如下

```

=====
请输入你的选择：5
请输入学号（12位数字）：202500120055
学生信息删除成功！可以通过菜单 9 撤销本次删除。
    
```

显示所有学生信息

=====

请输入你的选择：2

----- 第 1 个学生 -----

学号：202400130013
姓名：小明
年龄：20
性别：男
年级：大二
学院：信息科学与工程学院
专业：崇信学堂
班级：无需填写
宿舍门牌号：S2B115
手机号：18756559899
课程成绩：
 数据结构：96.00
 大学物理：92.00
 数字电子技术：99.00
 计算机组成原理：78.00
平均成绩：91.25

----- 第 2 个学生 -----

学号：202400120045
姓名：刘哲旭
年龄：20
性别：男
年级：大二
学院：信息科学与工程学院
专业：通信工程
班级：2班
宿舍门牌号：S1B207
手机号：15765477177
课程成绩：
 数字信号处理：98.00
 FPGA：95.00
 随机信号分析：99.00
 高频电子线路：100.00
平均成绩：98.00

撤销上一次删除

=====

请输入你的选择：9

撤销删除成功，学生信息已恢复。

----- 第 1 个学生 -----

学号: 202500120055
姓名: 小明
年龄: 19
性别: 男
年级: 大一
学院: 信息科学与工程学院
专业: 大一暂未分专业
班级: 大一暂未分班
宿舍门牌号: S1B411
手机号: 15787659898
课程成绩:
 体育: 99.00
 大学英语: 100.00
 大学物理: 87.00
 高等数学: 95.00
平均成绩: 95.25

----- 第 2 个学生 -----

学号: 202400130013
姓名: 小明
年龄: 20
性别: 男
年级: 大二
学院: 信息科学与工程学院
专业: 崇信学堂
班级: 无需填写
宿舍门牌号: S2B115
手机号: 18756559899
课程成绩:
 数据结构: 96.00
 大学物理: 92.00
 数字电子技术: 99.00
 计算机组成原理: 78.00
平均成绩: 91.25

----- 第 3 个学生 -----

学号: 202400120045
姓名: 刘哲旭
年龄: 20
性别: 男
年级: 大二
学院: 信息科学与工程学院
专业: 通信工程
班级: 2班
宿舍门牌号: S1B207
手机号: 15765477177
课程成绩:
 数字信号处理: 98.00
 FPGA: 95.00
 随机信号分析: 99.00
 高频电子线路: 100.00
平均成绩: 98.00

（七）为已有学生添加课程成绩

核心代码：“为已有学生添加课程成绩”的核心代码主要在 `student.c` 里的 `addGradeForStudent()`，它会先按学号查找学生，找到后再调用 `inputGrades()` 录入课程成绩。对应的成绩录入核心逻辑是 `student.c` 的 `inputGrades()`：

系统先输入学生学号，并通过哈希表查找该学生；如果学生不存在，则提示无法添加成绩；如果查找成功，则进入成绩录入循环，每输入一门课程和成绩，就创建一个成绩结点，并将其插入到该学生的成绩链表中。

```
=====
请输入你的选择：6
请输入学号（12位数字）：202400120045
正在为学生 刘哲旭 添加课程成绩。
请输入科目名称，输入 0 结束成绩录入：数据结构
请输入 数据结构 的成绩：88
成绩录入成功！
请输入科目名称，输入 0 结束成绩录入：0
成绩录入结束。
```

```
----- 第 5 个学生 -----
学号：202400120045
姓名：刘哲旭
年龄：20
性别：男
年级：大二
学院：信息科学与工程学院
专业：通信工程
班级：2班
宿舍门牌号：S1B207
手机号：15765477177
课程成绩：
    数据结构：88.00
    数字信号处理：98.00
    FPGA：95.00
    随机信号分析：99.00
    高频电子线路：100.00
    平均成绩：96.00
```

（八）成绩排序与统计

核心代码：

排序功能：`student.c` 的 `sortByAverageScore()` 和 `student.c` 的 `quickSort()`。

统计功能：`student.c` 的 `statistics()`。

排序时先将链表中的学生结点指针存入数组，再使用快速排序按照平均成绩降序排列。统计功能通过遍历学生链表，计算总平均分，并找出平均成绩最高和最低的学生。

按平均成绩从高到低排序结果：

第 1 名：

学号：202400120045

姓名：刘哲旭

年龄：20

性别：男

年级：大二

学院：信息科学与工程学院

专业：通信工程

班级：2班

宿舍门牌号：S1B207

手机号：15765477177

课程成绩：

数字信号处理：98.00

FPGA：95.00

随机信号分析：99.00

高频电子线路：100.00

平均成绩：98.00

第 2 名：

学号：202500120055

姓名：小明

年龄：19

性别：男

年级：大一

学院：信息科学与工程学院

专业：大一暂未分专业

班级：大一暂未分班

宿舍门牌号：S1B411

手机号：15787659898

课程成绩：

体育：99.00

大学英语：100.00

大学物理：87.00

高等数学：95.00

平均成绩：95.25

第 3 名：

学号：202400130013

姓名：小明

年龄：20

性别：男

年级：大二

学院：信息科学与工程学院

专业：崇信学堂

班级：无需填写

宿舍门牌号：S2B115

手机号：18756559899

课程成绩：

数据结构：96.00

大学物理：92.00

数字电子技术：99.00

计算机组成原理：78.00

平均成绩：91.25

=====

请输入你的选择：8

学生总人数：3

全体学生平均成绩：94.83

平均成绩最高：刘哲旭，学号：202400120045，平均分：98.00

平均成绩最低：小明，学号：202400130013，平均分：91.25

七、实验心得

通过本次大作业，我对数据结构在实际程序中的应用有了更直观的认识。以前学习链表、哈希表、栈和排序算法时，更多是停留在概念和单独练习上，而这次通过学生信息管理系统，我把这些知识结合到一个完整项目中，体会到了它们在实际功能中的作用。

在编写程序的过程中，我发现一个系统不仅仅是把功能写出来，还要考虑很多细节，比如输入是否合法、学号是否重复、删除后如何恢复、退出程序时如何释放内存等。这些问题让我更加理解 C 语言中指针和动态内存管理的重要性。

本次实验让我收获较大的是模块化设计思想。程序将不同功能分成多个文件实现，使代码更加清晰，也方便查找问题和后期修改。同时，通过实现按学号查找、平均成绩排序、成绩统计和撤销删除等功能，我进一步掌握了链表遍历、哈希查找、快速排序和栈操作的基本方法。

虽然在调试过程中遇到了一些问题，但通过不断修改和测试，最终系统能够较完整地运行。通过这次实验，我提高了综合运用数据结构知识解决实际问题的能力，也认识到写程序时需要注重代码结构、运行结果和用户体验。